

Cyber Security Internship Report

Week 5 - Task 14 (EH)

Cross-Site Scripting (XSS) - DVWA

Submitted By	MANOJ
Role	Cyber Security Intern
Internship Organisation	TechBiz Security
Internship ID	TBS-INT-2026-518
Submitted Date	19 SEPT 2026

TABLE OF CONTENTS

1. INTRODUCTION.....	3
2. OBJECTIVES.....	3
3. LAB ENVIRONMENT.....	3
4. REFLECTED XSS.....	3
5. STORED XSS.....	4
6. REFLECTED XSS VS STORED XSS.....	5
7. SECURITY IMPACT.....	6
8. ROOT CAUSE.....	6
9. REMEDIATION.....	6
10. TRYHACKME TRAINING.....	7
11. PRACTICAL FINDINGS.....	8
12. KEY LEARNINGS.....	8
13. CONCLUSION.....	8
14. EVIDENCE SUMMARY.....	8

1. INTRODUCTION

Cross-Site Scripting (XSS) is a web application security vulnerability in which untrusted input can result in client-side script execution in a user's browser when proper security controls are missing. The browser receives the affected page from the legitimate application and therefore treats the injected script as part of that site, which allows it to run with the privileges of the page it appears in.

This report documents Week 5 – Task 14 (EH). The task required practising Reflected XSS and Stored XSS on Damn Vulnerable Web Application (DVWA), observing JavaScript execution through a browser alert, understanding the security impact, and completing the TryHackMe XSS room. All practical testing was performed in a local laboratory environment. Throughout the report, general security concepts are presented separately from the practical observations that are visible in the captured screenshots.

2. OBJECTIVES

1. Understand Cross-Site Scripting.
2. Understand Reflected XSS and Stored XSS.
3. Perform XSS testing in a controlled DVWA laboratory environment.
4. Observe JavaScript execution through a browser alert.
5. Understand the difference between Reflected and Stored XSS.
6. Understand basic XSS remediation techniques.
7. Complete the TryHackMe XSS learning room.

3. LAB ENVIRONMENT

The following details are limited to what is visible in the screenshots or stated in the task information.

Item	Detail
Application	Damn Vulnerable Web Application (DVWA)
Environment	Local laboratory environment
Target URL shown in screenshots	127.0.0.1:8080/DVWA
Reflected XSS page	http://127.0.0.1:8080/DVWA/vulnerabilities/xss_r/
Stored XSS page	http://127.0.0.1:8080/DVWA/vulnerabilities/xss_s/
Browser	Mozilla Firefox
TryHackMe room	XSS (tryhackme.com/room/axss)
DVWA version, DVWA security level, operating system	Not demonstrated in the provided evidence.

4. REFLECTED XSS

4.1 Concept (general)

Reflected XSS occurs when an application includes data from the current HTTP request, such as a query-string parameter or form field, in its immediate response without adequate output encoding. If that data contains script, the browser executes it when the response is rendered. The payload is not stored by the application; it exists only in the request and its response, so exploitation normally depends on the victim submitting a crafted request, for example by following a specially constructed link.

4.2 Practical Demonstration (observed)

- The DVWA page titled “Vulnerability: Reflected Cross Site Scripting (XSS)” was accessed, and the XSS (Reflected) entry is highlighted in the DVWA menu.
- The address bar shows `http://127.0.0.1:8080/DVWA/vulnerabilities/xss_r/?name=<script>alert(1)<%2Fscript>#`. The payload `<script>alert(1)</script>` is carried in the name parameter; `%2F` is the URL-encoded form of the forward slash.
- The browser displayed an alert dialog from 127.0.0.1:8080 containing the value 1.
- The page output line beneath the “What’s your name?” input reads “Hello” with no visible text after it, which is consistent with the supplied value being processed as script rather than displayed as text.

Result. The alert demonstrates that the script supplied through the reflected input was executed by the browser. This confirms reflected script execution in the controlled DVWA laboratory. The screenshot does not show whether the value was typed into the form or entered directly in the address bar, and it does not show the DVWA security level. No further exploitation steps are claimed.

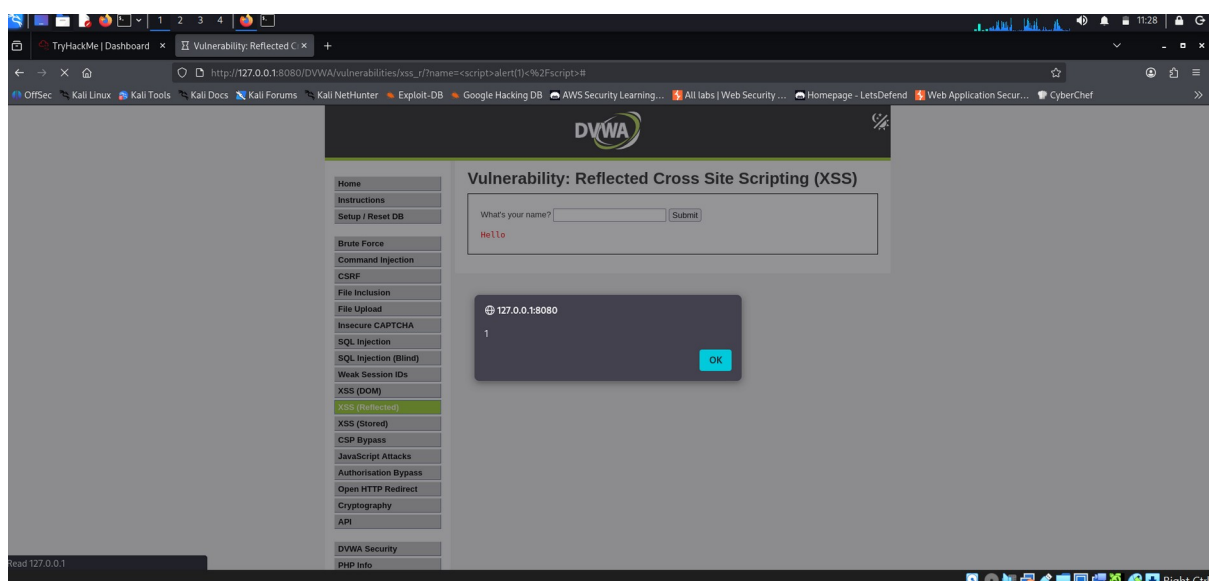


Figure 1: DVWA Reflected XSS payload execution resulting in a JavaScript alert.

5. STORED XSS

5.1 Concept (general)

Stored XSS, also called persistent XSS, occurs when a malicious script submitted to the application is saved on the server side, for example in a database, and is later included in pages returned to users without adequate output encoding. Typical locations are comments, guestbooks, forum posts, and profile fields. Because the payload is retained by the application, it can execute each time an affected page is loaded, without the victim needing to follow a crafted link.

5.2 Practical Demonstration (observed)

- The DVWA page titled “Vulnerability: Stored Cross Site Scripting (XSS)” was accessed at `/DVWA/vulnerabilities/xss_s/`, and the XSS (Stored) entry is highlighted in the DVWA menu.
- The page presents the guestbook form with the buttons Sign Guestbook and Clear Guestbook.
- The Name field contains `test`.
- The Message field contains the XSS script payload `<script>alert(1)</script>`.
- The browser displayed an alert dialog from 127.0.0.1:8080 containing the value 1.

Result. This demonstrates that script content entered into the stored input (the guestbook message field) was executed in the browser within the DVWA laboratory environment. The screenshot does not show the guestbook entry being displayed again after a page reload, or being viewed from a different

user or session. Persistence across sessions or users is therefore not demonstrated in the provided evidence.

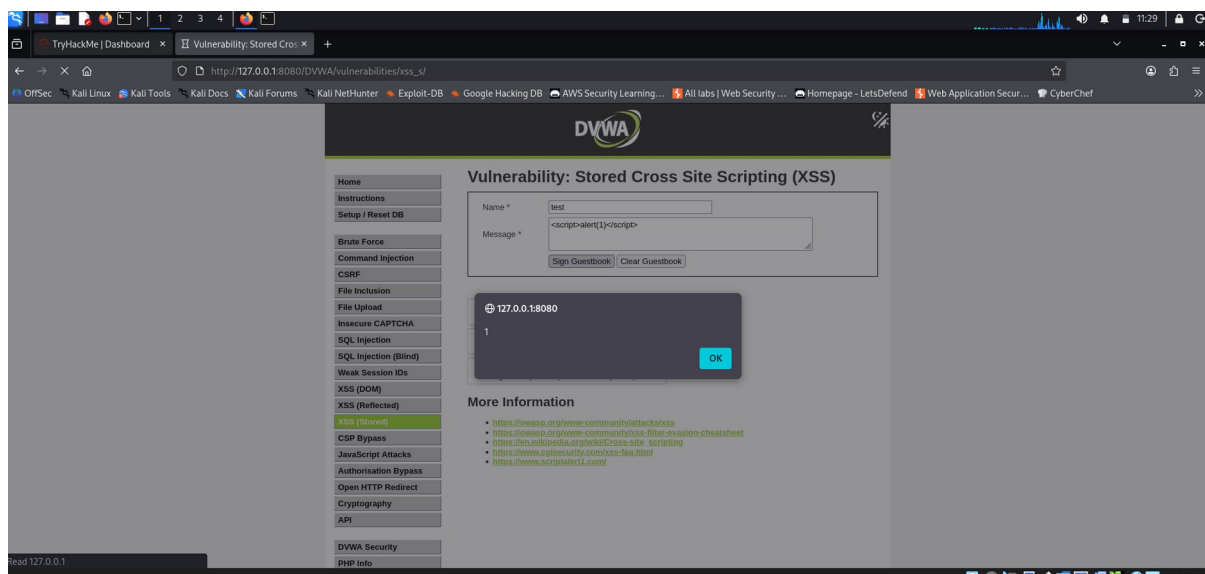


Figure 2: DVWA Stored XSS payload execution through the guestbook message field.

6. REFLECTED XSS VS STORED XSS

The table below compares the two types in general terms and relates each to the corresponding practical in this task.

Aspect	Reflected XSS	Stored XSS
Where the payload is supplied / stored	Supplied in the request (for example a URL parameter or form field). It is not stored by the application.	Submitted through an input such as a comment or guestbook and saved by the application.
How execution occurs	The application includes the request data in its immediate response, and the browser executes the script when it renders that response.	The application includes the saved content in a page it later serves, and the browser executes the script when that page is loaded.
Typical delivery mechanism	A crafted link or request that the victim is induced to open or submit.	The victim simply views the affected page; no crafted link is needed.
Persistence	Non-persistent. The payload exists only for that request.	Persistent until the stored content is removed.
Example from this practical	DVWA reflected input produced the alert (Figure 1).	DVWA guestbook/comment input produced the alert (Figure 2).
Security consideration	Depends on the victim being led to submit the crafted request; the payload is often visible in the URL.	Can affect every user who views the page, so it is generally considered to have wider reach and needs prompt removal of stored content in addition to fixing the code.

The persistence and reach described for Stored XSS are general characteristics. Only the alert execution was demonstrated in the practical evidence.

7. SECURITY IMPACT

In general, a successful XSS attack allows attacker-controlled JavaScript to run in a victim's browser in the context of the vulnerable site. Depending on the application and its controls, potential impacts include:

- Unauthorized JavaScript execution: script runs in the victim's browser without the user or the application intending it.
- Manipulation of webpage content: the script can alter what the page displays, for example inserting misleading text or forms.
- User interaction abuse: the script can act on the page on behalf of the user, such as triggering actions the user did not intend.
- Session-related attacks: where session identifiers or tokens are accessible to client-side script, they may be exposed or misused. Controls such as HttpOnly cookies reduce this risk.
- Phishing or malicious redirection: the page can be altered to present fake login prompts or to send the user to another site.
- Exposure of sensitive client-side information: data displayed on or held by the page may be readable by the script, depending on application controls.

Scope of the practical evidence. The screenshots in this report demonstrate JavaScript execution through an alert in the controlled DVWA laboratory environment. They do not demonstrate compromise of a real production system, and no session, data, or account impact was tested or claimed.

8. ROOT CAUSE

The following are common root causes of XSS vulnerabilities in general.

- Improper handling of untrusted input: data from users, URLs, or other external sources is treated as trustworthy.
- Failure to encode output properly: user-controlled data is written into a page without converting special characters for the context in which it appears.
- Inadequate input validation: input is not checked against what the application expects, so unexpected content such as script markup is accepted.
- Unsafe rendering of user-controlled content: content is inserted into the page in a way that the browser interprets as markup or script instead of plain text.
- Missing or insufficient browser-side security controls: controls such as a Content Security Policy are absent or too permissive, so injected script is not restricted.

These are general causes. The DVWA source code was not reviewed as part of this task, so none of them is claimed as specifically verified for the DVWA pages tested.

9. REMEDIATION

Output encoding and proper handling of untrusted input are the key defenses against XSS. The other measures below add further layers of protection.

- Context-aware output encoding: encode data according to where it is placed (HTML body, HTML attribute, JavaScript, or URL) so that the browser treats it as text. For example, in an HTML body context the characters < and > are converted to < and >.
- Input validation: accept only input that matches the expected type, format, and length, and reject or handle everything else. Validation supports but does not replace output encoding.
- HTML sanitization where HTML input is intentionally allowed: if the application must accept HTML, use a well-maintained sanitization approach that permits only a safe set of elements and attributes.

- Safe templating and framework APIs: use frameworks and templates that encode output by default, and avoid bypassing their protections.
- Avoid unsafe DOM manipulation: avoid inserting untrusted data with APIs that interpret it as HTML or script, and prefer APIs that treat it as text.
- Content Security Policy (CSP): implement an appropriate policy as an additional defense that limits which scripts the browser may run. It should not be relied on as the only control.
- Secure coding practices: treat all external data as untrusted, apply the measures consistently across the application, and follow secure coding guidance for the framework in use.
- Security testing during development: include code review and security testing in the development process so that XSS issues are found before release.

Application to this practical. For both scenarios tested, encoding the reflected name value and the stored guestbook message at output time would cause the browser to display the payload as text instead of executing it. This is a general principle and has not been tested against DVWA in this task.

10. TRYHACKME TRAINING

The TryHackMe XSS room was completed. The screenshot shows the following:

Item	Visible in screenshot
Room	XSS
Description	Explore in-depth the different types of XSS and their root causes.
Completion	Room completed: 100%
Completed tasks visible	Task 1 – Introduction; Task 2 – Terminology and Types; Task 3 – Causes and Implications

Each of the three visible tasks is marked with a completion tick. Any further task rows below Task 3 are not visible, and no task answers, flags, or task content are shown. Browser tabs for a DVWA DOM Based page and an OWASP XSS page are also open, but no DOM-based XSS testing is demonstrated in the provided evidence.

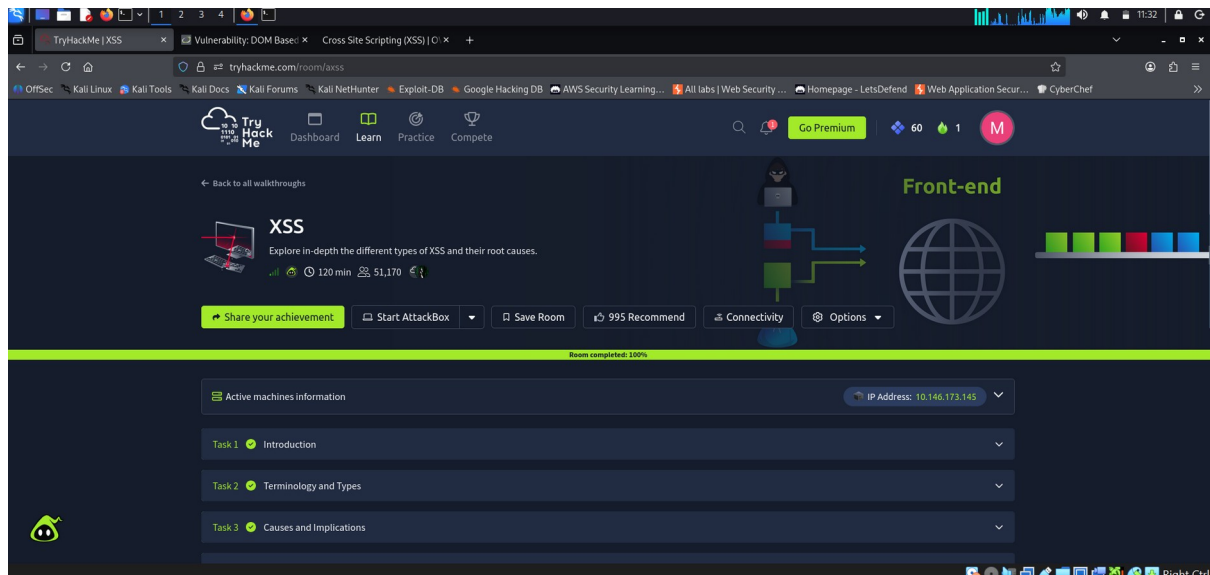


Figure 3: TryHackMe XSS room completed at 100%.

11. PRACTICAL FINDINGS

Finding	Vulnerability	Location	Evidence	Status
1	Reflected Cross-Site Scripting	DVWA Reflected XSS functionality	JavaScript alert displaying "1" (Figure 1)	Demonstrated in controlled DVWA laboratory
2	Stored Cross-Site Scripting	DVWA Stored XSS guestbook/comment functionality	JavaScript alert displaying "1" (Figure 2)	Demonstrated in controlled DVWA laboratory

No CVSS scores were calculated during this task, and none are assigned here.

12. KEY LEARNINGS

- The difference between Reflected XSS and Stored XSS, particularly in where the payload is held and how it reaches the browser.
- How unsafe input can result in browser-side JavaScript execution.
- The importance of output encoding and input validation.
- The importance of secure handling of user-controlled data.
- Practical use of DVWA for web application security testing in a controlled laboratory.
- Completion of the TryHackMe XSS training.

13. CONCLUSION

This task successfully demonstrated Reflected XSS and Stored XSS in the controlled DVWA environment, with browser-side JavaScript execution confirmed through alert dialogs displaying the value 1. Based on the provided evidence, the TryHackMe XSS room was completed at 100%. The practical results are limited to the laboratory environment and do not represent testing of a production system. The task reinforced the importance of secure input handling, context-aware output encoding, and appropriate defensive controls such as a Content Security Policy in protecting users from XSS.

14. EVIDENCE SUMMARY

Figure	Evidence	Section	Screenshot file
Figure 1	DVWA Reflected XSS payload execution	4. Reflected XSS	1_DVWA_Reflected_XSS_Payload_Execution.png
Figure 2	DVWA Stored XSS payload execution	5. Stored XSS	2_DVWA_Stored_XSS_Payload_Execution.png
Figure 3	TryHackMe XSS room completion at 100%	10. TryHackMe Training	3_THM_XSS_Room_Completion_.png